Class: Character						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
Constructor	1	Creates Zoro with valid name/alias/origin and positive wallet	"Zoro", "Pirate Hunter", "East Blue", 5000	Fields set as given; status = FREE	Fields set as given; status = FREE	**P**
	2	Falls back to "Unknown" when name/alias/origin are null	null, null, null, 50	name/alias/origin = "Unknown"	name/alias/origin = "Unknown"	**P**
	3	Falls back wallet to 0 and alias to "Unknown" on negative wallet/blank alias	"Zoro", " ", "East Blue", -20	alias = "Unknown"; wallet = 0	alias = "Unknown"; wallet = 0	**P**
setStatus	4	Clears devilFruitPower when Luffy's status becomes DEAD	luffy.setStatus(DEAD)	luffy.hasDevilFruit() = false	luffy.hasDevilFruit() = false	**P**
	5	Leaves devilFruitPower untouched on a non-DEAD status change	luffy.setStatus(CAPTURED)	luffy.hasDevilFruit() = true	luffy.hasDevilFruit() = true	**P**
	6	Ignores a null status argument	luffy.setStatus(null)	luffy.getStatus()	luffy.getStatus()	**P**
addWallet	7	Credits Zoro's wallet with a positive amount	zoro.addWallet(100)	wallet=5100; Returns true	wallet=5100; Returns true	**P**
	8	Rejects a zero amount	zoro.addWallet(0)	wallet unchanged; Returns false	wallet unchanged; Returns false	**P**
	9	Rejects a negative amount	zoro.addWallet(-20)	wallet unchanged; Returns false	wallet unchanged; Returns false	**P**
deductWallet	10	Debits an amount smaller than the balance	zoro.deductWallet(30) (wallet=5100)	wallet=5070; Returns true	wallet=5070; Returns true	**P**
	11	Debits an amount exactly equal to the balance	zoro.deductWallet(5070)	wallet=0; Returns true	wallet=0; Returns true	**P**
	12	Rejects an amount exceeding the balance	zoro.deductWallet(100) (wallet=0)	wallet unchanged; Returns false	wallet unchanged; Returns false	**P**
setDevilFruitPower	13	Links an unfruited character to a fruit	zoro.setDevilFruitPower(Yami Yami no Mi, unowned)	link set; Returns true	link set; Returns true	**P**
	14	Rejects a null fruit	zoro.setDevilFruitPower(null)	Returns false	Returns false	**P**
	15	Rejects assignment when the character already owns a fruit	luffy.setDevilFruitPower(Mera Mera no Mi) (Luffy already owns Gomu Gomu no Mi)	Returns false; original retained	Returns false; original retained	**P**

Class: Pirate						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
Constructor	1	Creates Luffy with valid bounty and role	"Monkey D. Luffy", "Straw Hat", "East Blue", 100, 300000000, "Captain"	bounty = 300000000; role = "Captain"	bounty = 300000000; role = "Captain"	**P**
	2	Falls back bounty to 0 on a negative value	bounty = -100	bounty=0	bounty=0	**P**
	3	Falls back role to "Unassigned" on a blank string	pirateRole = ""	role = "Unassigned"	role = "Unassigned"	**P**
assignCrew	4	Links unaffiliated Zoro to a crew	zoro.assignCrew(strawHats)	Returns true	Returns true	**P**
	5	Rejects a null crew	zoro.assignCrew(null)	Returns false	Returns false	**P**
	6	Rejects re-linking an already-affiliated pirate	luffy.assignCrew(anotherCrew)	Returns false; original retained	Returns false; original retained	**P**
toggleCaptain	7	Promotes Luffy to captain once he has a crew	luffy.toggleCaptain(true)	isCaptain = true; role = "Captain"; Returns true	isCaptain = true; role = "Captain"; Returns true	**P**
	8	Demotes Luffy from captain	luffy.toggleCaptain(false)	isCaptain = false; role = "Unassigned"; Returns true	isCaptain = false; role = "Unassigned"; Returns true	**P**
	9	Rejects toggling captaincy on a pirate with no crew	buggy.toggleCaptain(true) (no crew)	no change; Returns false	no change; Returns false	**P**
performDuty	10	Prints role-specific text for a defined role (Captain)	luffy.performDuty() (role = "Captain")	"Let's get sailing!"	"Let's get sailing!"	**P**
	11	Prints role-specific text for a defined role (Cook)	luffy.performDuty() (role = "Cook")	"Some delicious stuff I'm cooking here."	"Some delicious stuff I'm cooking here."	**P**
	12	Prints role-specific text for a defined role (Navigator)	luffy.performDuty() (role = "Navigator")	"Onwards! Follow the map."	"Onwards! Follow the map."	**P**

Class: Marine						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
Constructor	1	Creates Smoker with rank VICE_ADMIRAL	"Smoker", "White Hunter", "Loguetown", 10000, VICE_ADMIRAL	Fields set as given; marineRank = VICE_ADMIRAL	Fields set as given; marineRank = VICE_ADMIRAL	**P**
	2	Falls back rank to ENSIGN on a null value	marineRank = null	Fields set as given; marineRank = ENSIGN	Fields set as given; marineRank = ENSIGN	**P**
	3	Accepts the highest enum rank as a boundary value	marineRank = FLEET_ADMIRAL	Fields set as given; marineRank = FLEET_ADMIRAL	Fields set as given; marineRank = FLEET_ADMIRAL	**P**
performDuty	4	Prints VICE_ADMIRAL-specific duty text	zoro.performDuty() (rank = VICE_ADMIRAL)	"Time to lead a buster fleet call!"	"Time to lead a buster fleet call!"	**P**
	5	Prints rank-specific text for the lowest rank	zoro.performDuty() (rank = ENSIGN)	"Following a superior's orders."	"Following a superior's orders."	**P**
	6	Prints rank-specific text for the highest rank	zoro.performDuty() (rank = FLEET_ADMIRAL)	"Each one of you! Man up and take charge!"	"Each one of you! Man up and take charge!"	**P**
promoteRank	7	Promotes a low-ranked marine to the next rank	tashigi.promoteRank() (rank = ENSIGN)	rank = LIEUTENANT; Returns true	rank = LIEUTENANT; Returns true	**P**
	8	Promotes a mid-ranked marine to the next rank	smoker.promoteRank() (rank = VICE_ADMIRAL)	rank = ADMIRAL; Returns true	rank = ADMIRAL; Returns true	**P**
	9	Rejects promotion when already at the highest rank	promoteRank() (rank = FLEET_ADMIRAL)	rank unchanged; Returns false	rank unchanged; Returns false	**P**
assignCorps	10	Links unaffiliated Smoker to Marine G-5	smoker.assignCorps(marineG5)	Returns true	Returns true	**P**
	11	Rejects a null corps	smoker.assignCorps(null)	Returns false	Returns false	**P**
	12	Rejects re-linking an already-affiliated marine	smoker.assignCorps(Marine HQ) (Smoker already in Marine G-5)	Returns false; original retained	Returns false; original retained	**P**

Class: Civilian						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
Constructor	1	Creates Makino with valid profession and residence	"Makino", "Makino", "Windmill Village", 5000, "Bartender", "Windmill Village"	Fields set as given	Fields set as given	**P**
	2	Falls back profession to "Unemployed" on a null value	profession = null	profession = "Unemployed"	profession = "Unemployed"	**P**
	3	Falls back residence to "Homeless" on a blank value	residence = " "	residence = "Homeless"	residence = "Homeless"	**P**
performDuty	4	Prints profession-specific text for a defined profession	makino.performDuty() (profession="Bartender")	"Any drinks you want?"	"Any drinks you want?"	**P**
	5	Prints profession-specific text for a different defined profession	scholarCivilian.performDuty() (profession="Scholar")	"What is it that you wanna know?"	"What is it that you wanna know?"	**P**
	6	Falls through to the default branch for an undefined profession	franky.performDuty() (profession="Shipwright's Apprentice")	"Working as the town's Shipwright's Apprentice..."	"Working as the town's Shipwright's Apprentice..."	**P**

Class: PirateHunter						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
Constructor	1	Creates Johnny with valid combat style and capture count	"Johnny", "Johnny", "East Blue", 200, "Swordsmanship", 3	Fields set as given	Fields set as given	**P**
	2	Falls back combat style to "None" on a null value	combatStyle = null	combatStyle = "None"	combatStyle = "None"	**P**
	3	Falls back confirmed captures to 0 on a negative value	confirmedCaptures = -5	confirmedCaptures = 0	confirmedCaptures = 0	**P**
setConfirmedCaptures	4	Overwrites the tally with a positive value	johnny.setConfirmedCaptures(20)	confirmedCaptures = 20	confirmedCaptures = 20	**P**
	5	Accepts zero as a valid boundary value	johnny.setConfirmedCaptures(0)	confirmedCaptures = 0	confirmedCaptures = 0	**P**
	6	Falls back to 0 on a negative value	johnny.setConfirmedCaptures(-3)	confirmedCaptures = 0	confirmedCaptures = 0	**P**
	7	Prints style-specific text for a defined combat style	johnny.performDuty() (combatStyle="Swordsmanship")	"I'll cut you down!"	"I'll cut you down!"	**P**
	8	Prints style-specific text for a different defined style	zoroHunter.performDuty() (combatStyle="Haki")	"Need to channel my haki. Hayah!"	"Need to channel my haki. Hayah!"	**P**
	9	Falls through to the default branch for an undefined style	genericHunter.performDuty() (combatStyle="Marksmanship")	"Fighting with Marksmanship..."	"Fighting with Marksmanship..."	**P**

Class: DevilFruit						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
Constructor	1	Creates Gomu Gomu no Mi with valid name/category/ability	"Gomu Gomu no Mi", PARAMECIA, "Rubber body"	Fields set as given	Fields set as given	**P**
	2	Falls back category to UNDETERMINED on a null value	category = null	category = UNDETERMINED	category = UNDETERMINED	**P**
	3	Falls back fruit name to "Unknown" on a blank value	fruitName = ""	fruitName = "Unknown"	fruitName = "Unknown"	**P**
assignNewOwner	4	Assigns Gomu Gomu no Mi to Luffy (no prior fruit)	gomuGomu.assignNewOwner(luffy)	Returns true	Returns true	**P**
	5	Rejects assignment when the fruit already has an owner	gomuGomu.assignNewOwner(zoro) (still owned by Luffy)	Returns false	Returns false	**P**
	6	Rejects assignment to a DEAD character	meraMera.assignNewOwner(ace) (Ace is DEAD)	Returns false	Returns false	**P**
	7	Rejects assignment to a character who already owns a different fruit	meraMera.assignNewOwner(luffy) (Luffy already owns Gomu Gomu no Mi)	Returns false	Returns false	**P**
displayFruit	8	Shows Luffy as current owner after assignment	gomuGomu.displayFruit()	"Current Owner: Monkey D. Luffy"	"Current Owner: Monkey D. Luffy"	**P**
triggerReincarnation	9	Clears current owner when Luffy dies	luffy.setStatus(DEAD)	"Current Owner: None"	"Current Owner: None"	**P**
	10	Adds Luffy to the fruit's historical owners	gomuGomu.getHistoricalOwners()	Contains Luffy	Contains Luffy	**P**
	11	No-ops when called again on an already-unowned fruit	gomuGomu.triggerReincarnation() (2nd call)	historicalOwners list unchanged	historicalOwners list unchanged	**P**

Class: PirateCrew						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
recruitMember	1	Recruits Luffy as founding captain	new PirateCrew("Straw Hat Pirates", "Going Merry", luffy)	crewMembers.size() = 1, captain = Luffy	crewMembers.size() = 1, captain = Luffy	**P**
	2	Recruits Zoro into Straw Hat Pirates	strawHats.recruitMember(zoro)	Returns true	Returns true	**P**
	3	Rejects Zoro re-recruiting into a second crew	buggyPirates.recruitMember(zoro)	Returns false	Returns false	**P**
setCaptain	4	Sets Zoro as captain once he is a crew member	strawHats.setCaptain(zoro)	Returns true (Luffy demoted, Zoro promoted)	Returns true (Luffy demoted, Zoro promoted)	**P**
	5	Rejects a null captain argument	strawHats.setCaptain(null)	Returns false	Returns false	**P**
	6	Rejects a pirate who isn't a member of this crew	strawHats.setCaptain(buggy)	Returns false	Returns false	**P**
goodbyeMember	7	Removes a regular member and clears their crew reference	strawHats.goodbyeMember(zoro) (non-captain)	Returns true	Returns true	**P**
	8	Vacates the captain's seat when the departing member is captain	strawHats.goodbyeMember(luffy) (captain)	true; captain = null	true; captain = null	**P**
	9	Rejects removing a pirate who isn't a member	strawHats.goodbyeMember(buggy)	Returns false	Returns false	**P**
getTotalBounty	10	Sums bounties of Luffy (300,000,000) + Zoro (111,000,000), both FREE	strawHats.getTotalBounty()	411000000	411000000	**P**
	11	Excludes Zoro's bounty after he is captured	zoro.setStatus(CAPTURED) getTotalBounty()	300000000	300000000	**P**
	12	Returns 0 once the crew has no contributing members left	after all members removed/captured/dead	0	0	**P**
displayCrewInfo	13	Prints crew summary	strawHats.displayCrewInfo()	Displays the proper crew information	Displays the proper crew information	**P**

Class: MarineCorps						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
Constructor	1	Stores operational funds provided at creation	"Marine G-5", "New World", "Doberman", 500000	getOpFunds() = 500000	getOpFunds() = 500000	**P**
	2	Falls back name/base to defaults on null values	corpsName = null, baseLocation = null	"Unnamed Corps" & "Unknown Base Location"	"Unnamed Corps" & "Unknown Base Location"	**P**
	3	Falls back operational funds to 0 on a negative value	opFunds = -500	getOpFunds() = 0	getOpFunds() = 0	**P**
recruitMarine	4	Recruits Smoker into Marine G-5	marineG5.recruitMarine(smoker)	Returns true	Returns true	**P**
	5	Rejects a null marine	marineG5.recruitMarine(null)	Returns false	Returns false	**P**
	6	Rejects re-recruiting an already-affiliated marine	marineG5.recruitMarine(smoker) again	Returns false	Returns false	**P**
goodbyeMember	7	Removes a current member and clears their corps reference	marineG5.goodbyeMember(smoker)	Returns true	Returns true	**P**
	8	Rejects removing a marine who isn't a member of this corps	marineG5.goodbyeMember(tashigi) (Tashigi belongs to a different corps object)	Returns false	Returns false	**P**
addOpFunds	9	Credits the corps with a positive amount	marineG5.addOpFunds(5000)	funds+5000; Returns true	funds+5000; Returns true	**P**
	10	Rejects a zero amount	marineG5.addOpFunds(0)	unchanged; Returns false	unchanged; Returns false	**P**
	11	Rejects a negative amount	marineG5.addOpFunds(-100)	unchanged; Returns false	unchanged; Returns false	**P**
displayMarineInfo	12	Prints corps summary with correct funds and unit size	marineG5.displayMarineInfo()	Displays the proper corps information	Displays the proper corps information	**P**

Class: CharacterDatabase						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
createPirate / createMarine / createCivilian / createPirateHunter	1	Registers all 5 created characters	Luffy, Zoro, Smoker, Makino, Johnny	5 characters listed	5 characters listed	**P**
deleteCharacter	2	Deletes a crewed pirate holding a devil fruit	charDB.deleteCharacter(luffy.getCharacterID())	crew ref severed; fruit reincarnated & released; Returns true	crew ref severed; fruit reincarnated & released; Returns true	**P**
	3	Deletes a corps-affiliated marine with no fruit	charDB.deleteCharacter(smoker.getCharacterID())	corps ref severed; Returns true	corps ref severed; Returns true	**P**
	4	Rejects deleting an unregistered id	charDB.deleteCharacter(9999)	Returns false	Returns false	**P**
findCharacterByID	5	Finds an existing character by id	charDB.findCharacterByID(zoro.getCharacterID())	Returns Zoro	Returns Zoro	**P**
	6	Returns null for a nonexistent id	charDB.findCharacterByID(9999)	Returns null	Returns null	**P**

Class: AffiliationDatabase						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
createPirateCrew	1	Creates and registers a crew for an unaffiliated captain	affDB.createPirateCrew("Straw Hat Pirates", "Going Merry", luffy)	crew created; "Created Pirate Crew: Straw Hat Pirates"	crew created; "Created Pirate Crew: Straw Hat Pirates"	**P**
	2	Throws when the captain is already affiliated	affDB.createPirateCrew(...,luffy) (already a captain)	throws IllegalArgumentException	throws IllegalArgumentException	**P**
	3	Throws when the captain is null	affDB.createPirateCrew(...,null)	throws IllegalArgumentException	throws IllegalArgumentException	**P**
deletePirateCrew	4	Disbands a crew with multiple members	affDB.deletePirateCrew(strawHats.getCrewID())	every member's crew ref cleared; Returns true	every member's crew ref cleared; Returns true	**P**
	5	Rejects deleting a nonexistent crew id	affDB.deletePirateCrew(9999)	Returns false	Returns false	**P**
deleteMarineCorps	6	Dissolves a corps with members	affDB.deleteMarineCorps(marineG5.getCorpsID())	every member's corps ref cleared; Returns true	every member's corps ref cleared; Returns true	**P**
	7	Rejects deleting a nonexistent corps id	affDB.deleteMarineCorps(9999)	Returns false	Returns false	**P**

Class: DevilFruitDatabase						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
deleteDevilFruit	1	Deletes a fruit that is currently owned	fruitDB.deleteDevilFruit(gomuGomu.getFruitID())	owner's fruit ref cleared; Returns true	owner's fruit ref cleared; Returns true	**P**
	2	Deletes a fruit that is currently unowned	fruitDB.deleteDevilFruit(meraMera.getFruitID())	Returns true	Returns true	**P**
	3	Rejects deleting a nonexistent fruit id	fruitDB.deleteDevilFruit(9999)	Returns false	Returns false	**P**

Class: BountyDatabase						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
registerCapture	1	Marine with a corps captures Zoro alive	bountyDB.registerCapture(zoro, smoker, false) (smoker in Marine G-5)	zoro.status = CAPTURED; reward credited to Marine G-5's opFunds; zoro.bounty=0	zoro.status = CAPTURED; reward credited to Marine G-5's opFunds; zoro.bounty=0	**P**
	2	Pirate Hunter kills Buggy	bountyDB.registerCapture(buggy, johnny, true)	buggy.status = DEAD; buggy removed from crew; reward to Johnny's wallet; Johnny's confirmedCaptures+1	buggy.status = DEAD; buggy removed from crew; reward to Johnny's wallet; Johnny's confirmedCaptures+1	**P**
	3	Rejects a pirate acting as captor	bountyDB.registerCapture(buggy, zoro, false)	throws InvalidCaptorException	throws InvalidCaptorException	**P**
	4	Rejects capturing a target that isn't FREE	bountyDB.registerCapture(zoro, smoker, false) (Zoro already CAPTURED)	throws IllegalArgumentException	throws IllegalArgumentException	**P**
	5	Rejects a null captor	bountyDB.registerCapture(buggy, null, false)	throws InvalidCaptorException	throws InvalidCaptorException	**P**
findCaptureById	6	Finds an existing capture record	bountyDB.findCaptureById(1)	Returns CaptureRecord with Id 1	Returns CaptureRecord with Id 1	**P**
	7	Returns null for a nonexistent id	bountyDB.findCaptureById(9999)	Returns null	Returns null	**P**

Class: FileDirectory						
Method	**#**	**Test Description**	**Sample Input Data**	**Expected Output**	**Actual Output**	**P/F**
saveCharacters	1	Saves a list with multiple character subtypes	saveCharacters([luffy, smoker, makino])	characters.txt overwritten, one record line per character	characters.txt overwritten, one record line per character	**P**
	2	Saves an empty roster	saveCharacters(emptyList)	file overwritten with only the header line	file overwritten with only the header line	**P**
	3	Overwrites prior contents on a second save call	saveCharacters(listB) after saveCharacters(listA)	2nd call's data fully replaces the 1st	2nd call's data fully replaces the 1st	**P**
archiveFile	4	Archives a file that exists on disk	FileDirectory.archiveFile("characters.txt") after a save	"characters_backup.txt" created; confirmation printed	"characters_backup.txt" created; confirmation printed	**P**
	5	Throws when the source file doesn't exist	FileDirectory.archiveFile("ghost_file.txt")	throws DataIOException	throws DataIOException	**P**
appendCapture	6	Appends entries to the capture log across multiple calls without erasing prior ones	appendCapture(record1), then appendCapture(record2)	capture_log.txt contains both lines, in order	capture_log.txt contains both lines, in order	**P**